



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD CIENCIAS DE LA COMPUTACIÓN

INGENIERÍA EN SOFTWARE

INFORME 3 GRUPO H

MATERIA:

INGENIERÍA DE REQUERIMIENTOS

INTEGRANTE:

NIEVES SANCHEZ JIMMY SAMUEL CI: 1250907878 - jnievess@uteq.edu.ec

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERÓN

CURSO/PARALELO:

CUARTO SOFTWARE B

QUEVEDO – LOS RÍOS – ECUADOR

PPA 2026 – 2027

Índice

1. Introducción	2
2. Datos generales de la exposición	2
3. Desarrollo de la exposición	2
3.1. Marco teórico: arquitectura MDA y niveles de abstracción	2
3.2. Modelo UML del subsistema y características de BOUML	3
3.3. Ventajas y limitaciones de BOUML	3
3.4. Demostración práctica	4
3.4.1. Creación del proyecto y del primer diagrama de clases	4
3.4.2. Generación de código sobre un módulo más completo	4
3.4.3. Edición de clases y verificación del proceso de regeneración	4
4. Análisis y conclusiones	5

1. Introducción

El presente informe documenta la exposición correspondiente al Grupo H dentro de la asignatura de Ingeniería de Requisitos, en la cual se presentó la herramienta CASE BOUML como recurso de apoyo al modelado UML y a la generación de código en distintos lenguajes de programación. La exposición estuvo a cargo de los integrantes Chavarría Tahiny, Mendoza Andy y Sánchez Gary, quienes distribuyeron los contenidos abordando respectivamente el marco teórico relacionado con la arquitectura MDA, el modelo UML del subsistema correspondiente al proyecto MundiPets, y las principales ventajas y limitaciones de la herramienta, además de una demostración práctica conjunta de sus funcionalidades. El objetivo del presente documento es reseñar de manera detallada los contenidos expuestos, con el fin de reforzar el proceso de aprendizaje sobre las herramientas orientadas al desarrollo dirigido por modelos.

2. Datos generales de la exposición

En la Tabla 1 se resumen los datos generales correspondientes a la exposición analizada, mientras que en la Tabla 2 se detalla la distribución de los temas abordados por cada integrante.

Grupo	H
Herramienta CASE	BOUML
Integrantes expositores	Chavarría Tahiny, Mendoza Andy, Sánchez Gary
Proyecto de referencia	MundiPets
Asignatura	Ingeniería de Requisitos
Docente	Ing. Guerrero Ulloa Gleiston Cicerón

Integrante	Tema abordado
Chavarría, Tahiny	Marco teórico sobre la arquitectura MDA y edición de clases con verificación del proceso de regeneración de código
Mendoza, Andy	Modelo UML del subsistema, características generales de BOUML y creación del proyecto con el primer diagrama de clases
Sánchez, Gary	Ventajas y limitaciones de la herramienta, y generación de código a partir de un módulo más completo del proyecto

3. Desarrollo de la exposición

3.1 Marco teórico: arquitectura MDA y niveles de abstracción

Chavarría inició la exposición explicando el motivo de la elección de BOUML como herramienta CASE y señalando que esta permite la generación de código en distintos lenguajes, entre ellos C++, el cual fue empleado como referencia durante la exposición. A continuación

desarrolló el marco teórico correspondiente a la arquitectura MDA, abordando sus niveles de abstracción: el modelo independiente de computación, el modelo independiente de plataforma y el modelo específico de plataforma, además de los conceptos de metamodelos y perfiles, elementos que sustentan el proceso de transformación de modelos a código dentro de este enfoque.

3.2 Modelo UML del subsistema y características de BOUML

Mendoza presentó el modelo UML del subsistema desarrollado para el proyecto MundiPets, describiendo las clases principales que lo componen: Usuario, Propietario, Veterinario, Mascota e Historial Médico. Explicó que la clase Usuario se especializa en las clases Propietario y Veterinario mediante una relación de generalización, y detalló cómo se modelaron los atributos, las operaciones, la visibilidad y la multiplicidad correspondientes a cada clase.

En cuanto a las características generales de la herramienta, señaló que BOUML permite realizar ingeniería inversa y generación de código de manera multiplataforma, soporta la traducción de diagramas UML a distintos lenguajes de programación, entre ellos C++, C, Java, Python, PHP y SQL, y cuenta con una interfaz sencilla, siendo además una herramienta de código abierto.

3.3 Ventajas y limitaciones de BOUML

Sánchez expuso las principales limitaciones de la herramienta, entre las que destacó una interfaz poco moderna, una documentación limitada y de difícil acceso, y una curva de aprendizaje pronunciada. Señaló también que BOUML realiza una generación parcial del código, es decir, únicamente genera determinadas partes de la clase y no su estructura completa, y que la integración con frameworks y bases de datos debe realizarse de forma manual. Adicionalmente advirtió que el proceso de regeneración de código resulta delicado, ya que una modificación seguida de una nueva generación puede eliminar el trabajo previamente realizado sobre esa clase. Como aspecto positivo indicó que la herramienta es gratuita, aunque cuenta con una comunidad de usuarios desactualizada.

Entre las ventajas mencionó una alta eficiencia en el modelado, el ahorro de tiempo que representa el uso de la herramienta, y la flexibilidad y el respaldo que ofrece su comunidad de usuarios. En la Tabla 3 se resumen estos aspectos.

Ventajas	Desventajas
Alta eficiencia en el modelado y ahorro de tiempo	Interfaz poco moderna y curva de aprendizaje pronunciada
Flexibilidad y respaldo de la comunidad de usuarios	Documentación limitada y de difícil acceso
Soporte multiplataforma y generación de código en varios lenguajes	Generación parcial del código, limitada a partes de la clase
Herramienta de código abierto y de uso gratuito	Regeneración delicada, con riesgo de pérdida del trabajo previo

3.4 Demostración práctica

3.4.1 Creación del proyecto y del primer diagrama de clases

Mendoza inició la demostración creando un nuevo proyecto en BOUML mediante la opción New, seleccionando la ruta de almacenamiento y guardando la configuración, lo que generó una carpeta contenedora de los archivos correspondientes al proyecto, el lenguaje de trabajo y los estereotipos disponibles. Posteriormente, mediante clic derecho sobre el proyecto, creó un diagrama de clases y accedió a la vista generada.

Sobre este diagrama creó la clase Propietario, a la cual asignó un atributo y una operación, y activó, desde las opciones de configuración de dibujo del diagrama, la visualización de los tipos de datos correspondientes a cada atributo. A continuación agregó la clase Mascota con su respectivo atributo y estableció entre ambas clases una relación de asociación. Como parte adicional de la demostración, elaboró también un diagrama de casos de uso.

Para la generación de código, Mendoza explicó que debía darse clic derecho sobre la vista de clases y, desde la opción Edit, crear un artefacto de tipo Source, procedimiento que debía repetirse clase por clase para que el código quedara disponible en la carpeta del proyecto. Finalmente abrió una terminal de PowerShell, se ubicó en dicha carpeta y ejecutó el archivo ejecutable generado por BOUML para comprobar su funcionamiento.

3.4.2 Generación de código sobre un módulo más completo

Sánchez retomó la demostración generando el código correspondiente a un módulo más elaborado de su proyecto de fin de curso, previamente preparado. Revisó los estereotipos asignados a cada una de las clases del diagrama, generó el código correspondiente y lo ejecutó mediante la consola de comandos, mostrando además el resultado en el editor Visual Studio Code para verificar su correcta generación.

3.4.3 Edición de clases y verificación del proceso de regeneración

Chavarría cerró la parte demostrativa editando un atributo de una clase ya existente mediante la opción de edición disponible al hacer clic derecho sobre dicho atributo, y agregó adicional-

mente un nuevo atributo y una nueva operación a la misma clase. Tras guardar los cambios, el sistema confirmó la actualización de cuatro clases del proyecto. Finalmente ejecutó nuevamente el código en la carpeta generada previamente por Mendoza para comprobar que las modificaciones realizadas se reflejaban correctamente, evidenciando así el funcionamiento del proceso de regeneración de código, aunque con las limitaciones señaladas previamente por Sánchez respecto a la delicadeza de este procedimiento.

4. Análisis y conclusiones

A partir de lo expuesto por el Grupo H se puede concluir que BOUML constituye una herramienta CASE flexible y eficiente para el modelado UML, particularmente valorada por su soporte multiplataforma y por su capacidad de generar código en múltiples lenguajes de programación a partir de un mismo diagrama de clases. No obstante, aspectos como la generación parcial del código, la documentación limitada y la delicadeza del proceso de regeneración constituyen limitaciones que deben considerarse al momento de integrar la herramienta dentro de un flujo de trabajo real. La demostración realizada sobre el proyecto MundiPets, que incluyó la creación de un diagrama de clases, la generación de código ejecutable y la edición posterior de una clase existente, permitió evidenciar de manera integral tanto las fortalezas como las limitaciones señaladas por el grupo, confirmando la utilidad de BOUML como herramienta de apoyo dentro del desarrollo dirigido por modelos.